CS 112

Introduction to Data Structures

WEEK 0

Intro to C++

Eric Araújo

Calvin University · Fall 2026

This Week

1. Structure of a C++ program
2. Compilation pipeline
3. Types, variables, I/O
4. Functions
5. Pointers **KEY**
6. Arrays
7. C++ vs. Python

Your First C++ Program

```cpp
int main() {


    return 0;
}
```

> Every C++ program needs a `main()` — this is where execution begins.

Your First C++ Program

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

Notice how `main()` morphed into its final form — we added the include, namespace, and output line.

Anatomy of a C++ Program

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

1 `#include <iostream>`

Pulls in the I/O library — C++'s version of `import`.

2 `using namespace std;`

Lets you write `cout` instead of `std::cout`.

5 `cout << … << endl;`

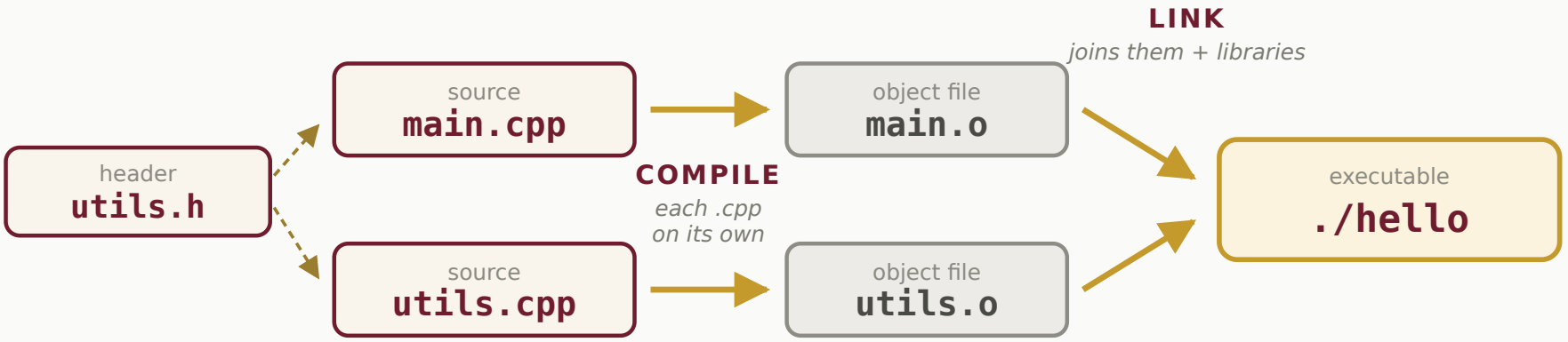
Prints to the terminal. `<<` is the "insertion" operator.

6 `return 0;`

Tells the OS the program finished cleanly.

Compilation Pipeline

headers are #included — pasted into each .cpp, never compiled on their own



COMPILE-TIME ERROR

```
error: 'cout' was not declared in
this scope
```

One file didn't make sense on its own — a typo, a missing `#include`, a type mismatch.

LINK-TIME ERROR

```
undefined reference to `helper()'
```

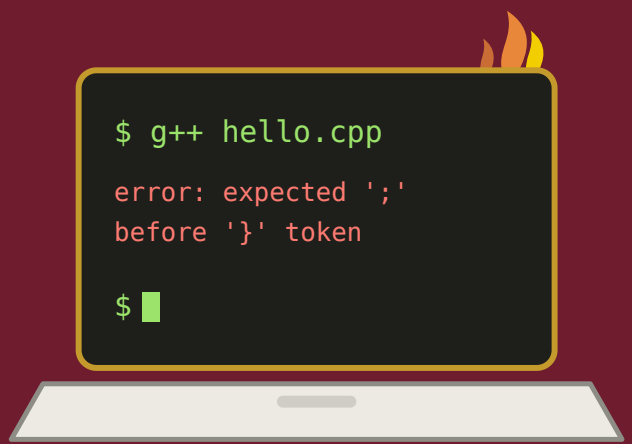
Every file compiled fine, but a function was *declared* and never *defined*.

Compilation: Key Points

- The compiler **translates** source code into machine-readable object files
- It also **type-checks** — many errors are caught before your program ever runs
- The linker **combines** object files and resolves references between them
- Libraries like `<iostream>` are pulled in during linking

> Unlike Python, you *must* compile before running. Upside: a whole class of bugs is caught at compile time, not in production.

Demo



Live coding. What could possibly go wrong?

(the semicolon is always missing on line 12)

TALK TO YOUR NEIGHBOR · TTYN

What does a compiler do?

A. Create an executable program from source code.

B. Check the source code for errors.

C. Pull in required libraries.

D. A and B only.

E. B and C only.

F. A, B, and C.

G. None of the above.

Data Types & Sizes

`int`	`unsigned`	`short`	`double`
4 bytes	4 bytes	2 bytes	8 bytes
−2B to 2B	*0 to 4B*	*−32K to 32K*	*15–16 digits*

`float`	`bool`	`char`	`string`
4 bytes	1 byte	1 byte	varies
6–7 digits	*true / false*	*'A', '\n'*	*"hello"*

```
cout << sizeof(int);       // 4   — check any type yourself
cout << sizeof(double);    // 8
cout << sizeof(grade);     // 1   — works on variables too
```

Types are **fixed at compile time**. Sizes can vary by machine — `sizeof` tells you the truth on yours.

Python vs. C++: Variables

Python

```
x = 7
```

namespace

x → **obj @ 0xABCD**

object space

0xABCD **int, value 7**

`x` is a *reference* to a typed object.

C++

```
int x = 7;
```

memory

0x12345678 **7** ← x

(4 bytes)

`x` *is* the memory location — it holds the value directly.

Declare, Then Initialize

```
int    count;              // declared — NOT initialized
int    total  = 0;         // declared and initialized
double pi     = 3.14159;
char   grade  = 'A';
bool   ready  = true;
string name   = "Ada";

cout << count;             // ⚠ undefined — could print anything
```

```
0x7ffd2a10  -8593204  ← count  (leftover bytes)
0x7ffd2a14      0     ← total  (you put it there)
```

In Python a name doesn't exist until you assign to it. In C++ the box exists the moment you *declare* it — holding whatever the last program left in that memory.

TALK TO YOUR NEIGHBOR · TTYN

Will this C++ code compile?

```
int x = 11;
x = "12";
```

A. Yes

B. No

C. Yes, but with warnings.

Using const

```
const int    MAX_SIZE = 100;
const double PI       = 3.14159265;

MAX_SIZE = 200;  // ❌ compiler error — read-only!
```

- Put `const` before the type to make a variable read-only
- The compiler **rejects** any code that tries to change it
- Convention: use `ALL_CAPS` names for constants

> Use `const` liberally — it documents intent and catches bugs at compile time.

Input / Output

```cpp
#include <iostream>
using namespace std;

int main() {
    double value;
    cout << "Enter a number: ";   // output → terminal
    cin  >> value;                // input  ← terminal
    cerr << "got " << value << endl; // error stream

    cout << "You entered: " << value << endl;
    return 0;
}
```

`cout` + `<<` — "insertion" (output to terminal)

`cin` + `>>` — "extraction" (input from terminal)

File I/O

```cpp
#include <iostream>
#include <fstream>
#include <cmath>
#include <cassert>
using namespace std;

int main() {
    ifstream fin("numbers.txt");
    assert(fin.is_open());   // stop if the file is missing

    double n;
    while (fin >> n) {       // false at end of file
        cout << sqrt(n) << endl;
    }
    fin.close();
    return 0;
}
```

`ifstream` reads from a file just like `cin` reads from the terminal. Use `ofstream` to write.

Decimal, Binary, Hexadecimal

```
int x = 33;         // decimal  (default)
int y = 0x21;       // hex:    2×16 + 1 = 33
int z = 0b100001;   // binary: 1×32 + 1 = 33

cout << x << " " << y << " " << z << endl;
// Output:  33 33 33
```

- `0x` prefix → hexadecimal (base 16)
- `0b` prefix → binary (base 2)
- Memory addresses are typically displayed in hex

Three ways to write the *same number*. The compiler stores them identically in memory.

Why bother with binary?

Everything on the machine is bits. Hex is just a friendlier way to read them.

Functions

Write the tests *before* the function — it forces you to think about what it should do:

```
assert(factorial(0) == 1);
assert(factorial(1) == 1);
assert(factorial(4) == 24);
assert(factorial(5) == 120);
```

This test-first style is called **test-driven development**. Your assertions become the spec.

Functions

```
unsigned factorial(unsigned n) {
    unsigned result = 1;
    for (unsigned i = 2; i <= n; ++i) {
        result *= i;
    }
    return result;
}

int main() {
    assert(factorial(0) == 1);
    assert(factorial(1) == 1);
    assert(factorial(4) == 24);
    assert(factorial(5) == 120);
    cout << "All tests passed!" << endl;
    return 0;
}
```

Pointers

A pointer is a variable that holds a **memory address**.

```
int y = 77;
```

0x12345678	**77**	← y

Pointers

Use `&` ("address-of") to get a variable's address:

```
int y    = 77;
int *x   = &y;   // x holds the address of y
```

0x12345678	**77**	← y
0x12345690	**0x12345678**	← x (pointer to y)

`int *x` declares x as a "pointer to int". Its value is an address, not 77.

Pointers: Dereferencing

Use `*` to get (or set) the value *at* the address:

```
int y  = 77;
int *x = &y;

cout << x;     // 0x12345678  — the address
cout << *x;    // 77          — follow the pointer
*x = 99;       // write through the pointer
cout << y;     // 99          — y itself changed
```

0x12345678	**99**	← y (changed via *x)
0x12345690	**0x12345678**	← x

TALK TO YOUR NEIGHBOR · TTYN

```
int y = 77;    // placed at address 0x12345678
int *x = &y;  // placed at address 0x12345690
cout << x;
```

What is printed?

A. 77

B. 0x12345678

C. 0x12345690

D. Error

TALK TO YOUR NEIGHBOR · TTYN

```
int y = 77;
int *x = &y;
cout << *y;   // note: *y, not *x
```

What is printed?

A. 77

B. 0x12345678

C. 0x12345690

D. Compile error

TALK TO YOUR NEIGHBOR · TTYN

```
int y = 77;
int *x = &y;
cout << *x;
```

What is printed?

A. 77

B. 0x12345678

C. 0x12345690

D. Error

TALK TO YOUR NEIGHBOR · TTYN

```
int y = 77;
int *x = &y;
*x = 78;
cout << *x;
```

What is printed?

A. 77

B. 78

C. 0x12345690

D. Error

TALK TO YOUR NEIGHBOR · TTYN

```
int y = 77;
int *x = &y;
*x = 78;
cout << y;    // note: y, not *x
```

What is printed?

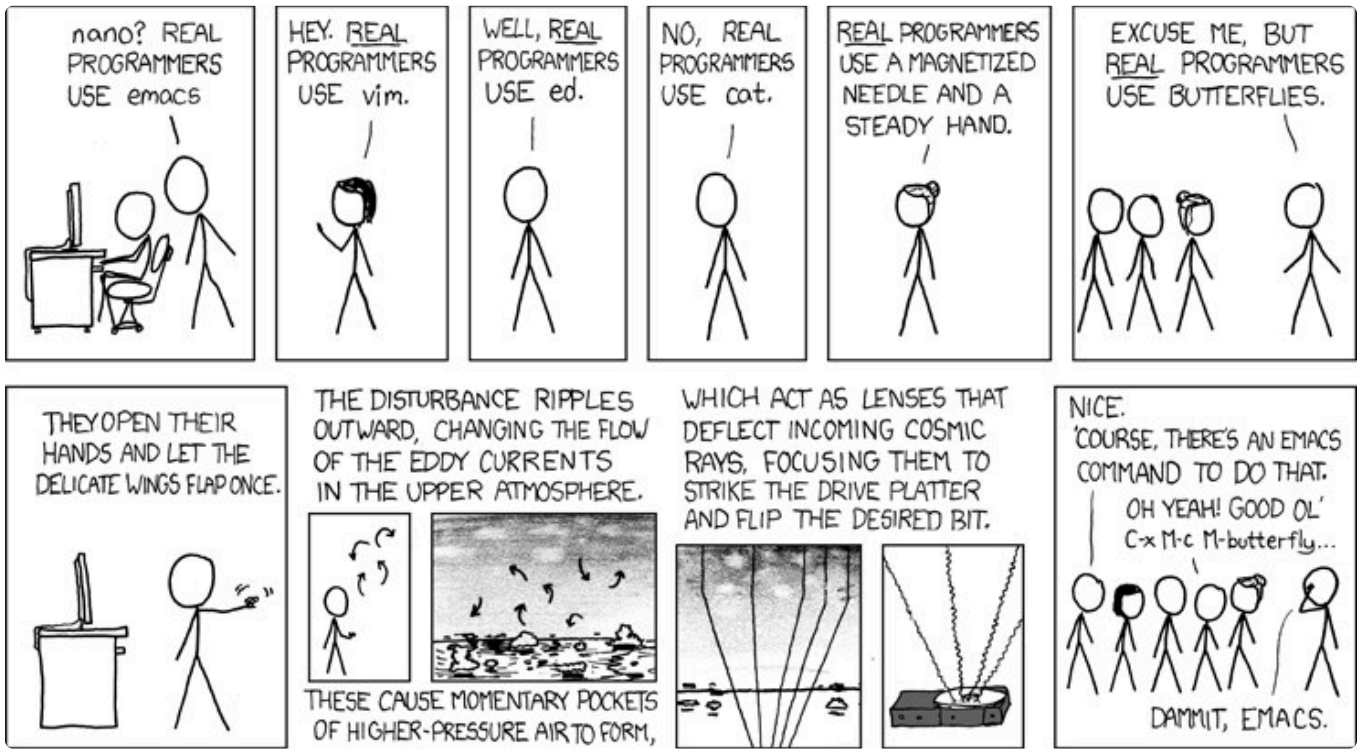
A. 77

B. 78

C. 0x12345690

D. Error

Meanwhile, in the C++ community...



You just spent five slides flipping bits by hand. Some people skip the pointer.

Arrays

Like Python lists, but **fixed size** and **no bounds checking**.

```
char  letters[26];              // 26 chars, uninitialized
float scores[10] = { 0 };       // all 0.0
int   vals[] = { 10, 9, 4, 2, 0 };  // size inferred: 5
```

- Size must be known at compile time (for stack arrays)
- Indexes: `0` to `n-1`
- Accessing out of bounds → **undefined behavior**, no crash guaranteed

Arrays: Memory Layout

```
int arr[6] = { 1, 3, 5, 2, 77, -11 };
```

All elements are **contiguous** in memory — no gaps:

index	[0]	[1]	[2]	[3]	[4]	[5]
base+0	**1**	**3**	**5**	**2**	**77**	**-11**

> The array name `arr` is a pointer to the first element — this links arrays and pointers directly.

Pointers & Arrays

```
int arr[] = { 3, -11, 4, 12 };
int *ptr  = &arr[2];   // ptr points to arr[2] → value 4

cout << *ptr;          // 4
cout << *(ptr + 1);    // 12   (next element)
cout << *(ptr - 1);    // -11  (previous element)
```

Pointer arithmetic lets you navigate arrays by address. This is how arrays *actually work* under the hood in C++. More on this in upcoming weeks.

TALK TO YOUR NEIGHBOR · TTYN

```
int arr[] = { 3, -11, 4, 12 };
int *ptr  = &arr[2];
cout << *ptr;
```

What is printed?

A. 3

B. 2

C. 4

D. -11

C++ vs. Python: Core Differences

Feature	Python	C++
Variables	Reference to a typed object	Named memory location with explicit type
Type checking	Dynamic (runtime)	Static (compile time)
Function return type	Implied from `return`	Declared explicitly
1D data structure	`list`	`array` or `vector`
Memory management	Automatic (garbage collected)	Mostly explicit
Entry point	First line executed	`main()`

C++ vs. Python: More Differences

Feature	Python	C++
Interface / impl split	Not explicit	`.h` header + `.cpp` source
Parameter passing	Pass-by-value (objects by ref)	By-value, by-reference, by-const-ref
Access control	Mostly none	`public` / `private`
Constructors	Called explicitly	Explicit *or implicit* (scope entry)
Destructors	None (GC handles it)	Defined; called implicitly at scope exit
Generics	None	Templates — powerful but complex
Strings	Built-in	`#include <string>`

Week 0 Recap

- C++ programs need a `main()` and must be **compiled** before running
- Types are **explicit and checked at compile time**
- Pointers hold **memory addresses** — the key concept in C++
- Arrays are contiguous memory blocks — fast, but no safety net
- C++ gives you more control than Python, and more responsibility

Next: Getting Started (pre-lab) · then a01 — RPG Character Creator →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**